Townlight Core Manual

Townlight Beta 1 · Naming candidate documentation edition 2 November 2026

Contents

- Using this manual
- Beta identity
- First run
- Daily workflow
- Connected workflows
- Architecture
- Shared catalog rules and scaffolding
- Technical specifications
- Configuration and administration
- API and integration examples
- MCP and automation boundary
- Security and permissions
- Backup recovery and maintenance
- Troubleshooting
- Verified workflows and limits
- Terms of service and license
- Glossary
- Alphabetical subject index
- Source and document control

Using this manual

Townlight Core helps IT administrators and platform maintainers. Core provides the shared contracts, ingestion, identity helpers, task infrastructure and recovery foundation used by Townlight. Staff encounter these capabilities through Setup, Settings and System Health. Business decisions stay with their owning product.

Staff should begin with First run and Daily workflow. Administrators should also read Architecture, Technical specifications, Configuration, Security and Recovery. Developers should use the exact API/data dictionary and examples. The table of contents and alphabetical subject index link to the relevant sections in the HTML edition; the PDF preserves headings and searchable text. An editable Word edition is included with the same documentation companion.

Beta identity

Identity	Value
Documentation	Townlight Beta 1 naming, Access and Notice

Identity	Value
	candidate
Intended installer version	1.7.0.1; build and acceptance pending
Product component	townlight-core 1.3.0.dev2
Principal import package	townlight_core
Native desktop source version	1.7.0-1
Candidate hashes	Pending candidate build, signing and installed acceptance
Current owned software license	AGPL-3.0-only
Acceptance composition	Fresh Windows x64 profile; inspections-preview; Station configured separately

The naming change advances this package's version and requires a new signed installer and installed acceptance. Those steps and publication remain pending. Historical tags, receipts and archived manuals keep their original names and certify only their identified artifacts. Use an empty database and fresh profile; no in-place legacy upgrade is supported. The approved Windows profile path remains `%LOCALAPPDATA%\CivicSuite`. Preserve existing profiles, data and recovery material.

First run

1. Obtain the identified signed private candidate and its corresponding-source companion from the beta coordinator. The landing pages in this documentation are not an installer download service.
2. On the intended fresh Windows test installation, verify the MSI hash and publisher signature before installing. Preserve unrelated profiles and recovery material. Installed acceptance for this source candidate remains pending; no in-place legacy upgrade is supported.
3. Launch Townlight under the intended named Windows account. Choose the storage folders. Select **Townlight Inspections candidate** (`inspections-preview`) for the bounded desktop composition, including Access and Notice. A narrower default selection will not expose all workspaces.
4. Enter the city profile, time zone and contacts. Create the first named local administrator, sign in and choose protected backup storage. **Continue without local AI** is valid for the deterministic workflows.
5. Run Health Check and wait for readiness. Open Townlight. In System Health select **Finish Records Migration** to activate the canonical Python/PostgreSQL runtime. Do not fabricate migration receipts or seed accounts.
6. Create named staff through Settings. Import authorized sources using each product's source library. All examples in this package are synthetic; they are not municipal authority.
7. Provision Station separately if it is part of the beta exercise. Selecting the desktop composition does not start or configure Station. Configure vision explicitly only if automatic photo drafting is selected.

Daily workflow

1. Complete setup

Choose folders and the full product composition. Create the city profile and the first named administrator. Choose a protected backup location. Defer optional local AI if it is not needed.

2. Activate the runtime

Run the first-run health check, wait for readiness and open Townlight. In System Health select Finish Records Migration to activate the canonical Python and PostgreSQL services. A running process alone is not a ready database.

3. Create staff access

Use Settings to create named staff. Sign out and confirm that staff can sign in with their own account. Keep administrator privileges with people responsible for setup; ordinary staff cannot administer users.

4. Import through the right product

Use the source library of Code, Zoning, Planning or Permitting. Inspect provenance and permissions there. Core ingestion helpers do not bypass the owning product’s source review.

5. Make a coherent backup

Save open work. Use Backup Now; it stops writers to preserve a consistent snapshot. Retain the backup receipt, then use Start to resume services.

6. Restore and verify

Use Restore Latest Backup after preserving current recovery material. A result of Restore needs service start is an intermediate state: use Start, then Check. Reopen a known saved record to confirm recovery.

7. Collect diagnostics

Record the time, selected composition, version, operation and exact error. Inspect System Health and create a redacted support bundle. Do not delete a profile as the first repair step.

Working safely

Use your own named account and save before leaving a screen. A saved draft is durable; unsaved form text may not be. If a write has an unknown result, reopen its record before repeating it.

Read source jurisdiction, dates, citation, version and visibility. A copied source hash identifies exact bytes, not their legal applicability. If a source changes or permission is removed, resolve the source problem rather than substituting stale cached content.

Keep originals and derivatives separate. Review the intended audience before export. A public summary excludes private fields only through the applicable copy contract and your review. A renamed private packet is still private content.

Drafting, review, approval, attachment, export, fulfillment and publication are separate operations. A Townlight workpaper does not itself adopt law, issue a permit, impose a fine or release a records response. The responsible staff member records the applicable decision.

Connected workflows

All products use selected Core contracts. Core does not create unrestricted access between products. Access and Notice provide the bounded staff workflows documented in their current manuals; installed acceptance remains pending.

Direction	Material
Input	Administrator setup choices, named staff accounts, source material submitted through the owning product, and protected backup storage.
Output	Ready local services, permission-aware shared contracts, task results, diagnostics and coherent backup or restore receipts.

Product boundary

Shared catalog, exemption and module-generator code exists in this private candidate. It is not a centrally hosted federation service. A business product still owns its decisions, identity mapping and permission checks. Historical placeholder ADRs are retained as history, rather than descriptions of the current implementation.

Architecture

Townlight Core architecture and evidence flow

Townlight Core architecture and evidence flow

The drawing shows the principal staff/evidence path. Native commands carry trusted identity to Python business services; PostgreSQL and retained original files hold durable state. A downstream copy does not silently transfer decision authority. Station has its own configured authority/media boundary, database and mutable files. The connected-workflow text supplies the drawing's reading order and accessibility alternative.

Shared catalog rules and scaffolding

These are existing Core developer and administrator contracts. They do not add a new Core administration screen. Staff use the owning product's normal controls; developers call the library only inside an authenticated application or an isolated synthetic test. The caller owns authorization and database commit or rollback.

Catalog import and readback

1. Review the catalog's `catalog_version`, domain names and systems. Each system supplies vendor name, protocol, data shape, record types, redaction tier 1–3 and optional discovery hints. Hints are limited to `ports`, `db_patterns` and `service_accounts`; do not put credentials in them.
2. In an authorized application transaction, call `townlight_core.catalog.import_catalog(session, bundle, actor_id=actual_actor_id)`. The actor is the authenticated application's identity. A changed or incomplete existing version is refused rather than overwritten. Use a newly reviewed version for changed catalog contents.
3. Read `domains(session)`, `system_catalog(session, domain)` and `connectors_for(session, domain)` before committing. Commit only the intended import; roll back on refusal. Repeating an identical complete version is a no-op.
4. Treat returned systems and connector recommendations as catalog information. Import does not discover vendor endpoints, install a connector or establish access to a city's system.

The following synthetic library example belongs inside an existing authorized async transaction. `session` and `actual_actor_id` are supplied by that application; this is not a command that seeds an administrator.

```
from townlight_core.catalog import import_catalog, system_catalog

bundle = {
    "catalog_version": "synthetic-1",
    "domains": [{"domain": "synthetic-records", "systems": [{
        "vendor_name": "Synthetic file source",
        "access_protocol": "file", "data_shape": "json",
        "common_record_types": ["synthetic minutes"],
        "redaction_tier": 2, "discovery_hints": {}
    }]}]
}
loaded = await import_catalog(session, bundle, actor_id=actual_actor_id)
rows = await system_catalog(session, "synthetic-records")
```

The Records compatibility facade reads its bundled `app/data/systems_catalog.json` through Core. Its existing administrator `POST /catalog/load` takes no replacement bundle body; `GET /catalog/domains`, `/catalog/systems?domain=...` and `/catalog/connectors?domain=...` expose readback. These non-desktop routes use the application's actual authenticated staff/admin dependencies. Do not substitute an invented role header. The shared SDK example above is the route for a developer-supplied bundle.

Jurisdictional rule evaluation and review

Select the jurisdiction and active reviewed rule versions before evaluating document text. Core's `Rule` carries `id`, `state_code`, `category`, `rule_type`, `definition` and integer `version`. Supported supplied rule types are `keyword` and `regex`; an unsupported rule or incomplete/timed-out scan raises `RuleEvaluationError`. An error must remain an incomplete scan, never "no exemptions found." Core does not assume a jurisdiction or claim complete legal coverage.

```
from uuid import UUID
from townlight_core.exemptions import Rule, evaluate, RuleEvaluationError

rule = Rule(
    id=UUID("11111111-1111-4111-8111-111111111111"),
    state_code="CO", category="Synthetic review flag",
    rule_type="keyword", definition="synthetic protected text", version=1
)
try:
    flags = evaluate("Contains synthetic protected text", "CO", rules=[rule])
except RuleEvaluationError:
    raise  # The owning application records an incomplete scan.
```

This is fictional test text, not a Colorado exemption rule. Read each result's matched text, rule identity/version and confidence, then use Records' explicit reviewer decision and legal/source basis. A flag is not an authorized withholding or redaction. Records' existing exemption rule/history/test and flag-review endpoints are listed in its technical reference; record changes through those controls, preserving history.

Scaffold plan and generate

Run the existing CLI from a developer environment containing the matching Core package. Choose an existing empty parent workspace directory, not an installed application's directory. The module's own directory will be created underneath it. The following creates only a fictional module project; it does not register, install, migrate or run it.

```
& $control -m townlight_core.scaffold plan civicnotes `
  --name 'Synthetic Notes' --description 'Synthetic scaffold exercise'
& $control -m townlight_core.scaffold generate civicnotes `
  --name 'Synthetic Notes' --description 'Synthetic scaffold exercise' `
  --target 'C:\TownlightSynthetic' --dry-run
```

Here `$control` is the Python executable in that isolated developer environment. Inspect the plan and files, then remove `--dry-run` only when that exact target is approved for generation. Existing files are refused, not silently overwritten. Exit 0 indicates success; refusal exits 2.

Options include `--version`, `--townlight_core-min-version`, `--permission-key`, repeated `--staff-role` and `--route-path`; inspect `--help` for the exact invocation. Keep generated access requirements and migration stubs under the new module's own review. No automatic product deployment follows generation.

Source contracts: `townlight_core/catalog/__init__.py`, `townlight_core/exemptions/__init__.py`, `townlight_core/scaffold/cli.py`, `project.py`, and the Records catalog facade.

Technical specifications

Runtime and storage

The accepted package targets **Windows x64 fresh installation**. Its native shell uses **Rust 2021, Tauri 2 and Microsoft WebView2**; browser assets are JavaScript built with **Vite 8.1.6**. This describes the native desktop frontend; Station has a separate React operator, specified below or in its product manual. Canonical business workflows use bundled **CPython 3.13.5, FastAPI, Pydantic, SQLAlchemy and Alembic**, with **PostgreSQL 17 / PostGIS** and managed original files. The installed runtime tested here contains FastAPI 0.162.0. Package minimum Python requirements can differ from the bundled executable; the metadata reference preserves both. No stack rewrite is implied by the beta label.

The native transport is fixed to loopback `127.0.0.1:15480`; PostgreSQL uses the configured local runtime port (default `15432`). Optional local model service defaults to `127.0.0.1:15434`. Those are deployment defaults, not internet endpoints. The runtime has health, module and readiness endpoints; readiness is stronger than process existence. The task queue is the existing local Core task infrastructure, not an assertion that a separate Redis service is required for this desktop composition.

The default native profile is `%LOCALAPPDATA%\CivicSuite`; the historical directory name remains part of compatibility. Do not rename it to Townlight. A configured `TOWNLIGHT_PROFILE_ROOT` must be absolute. Local configuration and secrets live under that profile; protected managed files and database state must be backed up coherently. Source/runtime identity is distinct from a product's display label.

Data and consistency

Named local accounts; native sessions; source and department authority; task receipts; profile configuration; coherent backup manifests; catalog, exemption and scaffold contracts.

Versioned saves use expected prior revision identifiers where defined. Reopen and reconcile on conflict. Exact exports and handoffs use source IDs, current visibility, revision and SHA256. Review belongs to the saved revision and does not transfer silently to edited text. Database changes use the owning product's migrations; a request or readiness check is not an authorization to mutate schema.

Capacity and platform statements

This beta has synthetic single-profile workflow evidence. It does not have a published benchmark for concurrent staff capacity, minimum RAM/CPU, large-source limits or a field SLA. Do not turn a developer machine's specification into a supported minimum. The new installer size remains pending its build; allow additional disk for extracted runtime, database, originals, exports, media and backups. Select equipment/storage after sizing those actual workloads.

The exact runtime payload lock records pinned files, sizes and hashes. The component metadata records shipped dependency declarations. These references are more precise than a manually copied dependency list. Third-party and historical license notices must accompany their matching payloads.

Configuration and administration

Native deployment configuration

Use Setup, Settings and System Health for supported staff operation. Administrators provision folders, backup storage, named accounts and the city profile. Select `inspections-preview` for the bounded desktop composition, including Access and Notice. Station remains a separate service boundary. Do not use a standalone product's demo server as a substitute for the installed workspace.

Technical deployments use `TOWNLIGHT_PRODUCT_PROFILE`, `TOWNLIGHT_PROFILE_ROOT`, `DATABASE_URL`, `TOWNLIGHT_RUNTIME_PORT`, `PORTAL_MODE` and the runtime-managed secret files. Treat database URLs and native session proofs as secrets. The installed native domain runtime explicitly refuses non-loopback binding. `PORTAL_MODE=private` is the accepted posture; changing it does not by itself commission an internet portal, authentication provider or reverse proxy.

The isolated documentation exporter sets a fictional profile and unusable synthetic database origin only to read schemas; those values are not deployment configuration. Keep deployment values in the intended process/account scope. An already-running process does not automatically inherit later environment edits.

API and integration examples

Native command example

The following JavaScript belongs in the existing trusted Townlight frontend/development environment with `invoke` imported from `@tauri-apps/api/core`. It is not a command for an arbitrary web page or an instruction for nontechnical staff. IDs must come from records created in an isolated synthetic exercise.

```
import { invoke } from '@tauri-apps/api/core';
const result = await invoke('get_app_state', {});
console.log(result);
```

Trusted local HTTP example

An administrator/developer can inspect an already-configured isolated runtime from a trusted local process. This PowerShell example reads the signed-in synthetic profile's existing proof into memory; it does not create a user, seed a session, reveal the proof in output or bypass a permission check. Do not use real city data in development examples. A read response does not prove approval or publication.

```
$profilePath = Join-Path $env:LOCALAPPDATA 'CivicSuite'
$sessionPath = Join-Path $profilePath 'config\local-session.json'
$session = Get-Content -LiteralPath $sessionPath -Raw | ConvertFrom-Json
$headers = @{ 'X-Townlight-Session' = $session.session_hash }
Invoke-RestMethod -Uri 'http://127.0.0.1:15480/desktop/v1/suite/state'
-Headers $headers
Remove-Variable session, headers
```

A synthetic write with subsequent readback

This example demonstrates the existing Meetings HTTP contract in an isolated exercise. It is intentionally fictional. It does not start a service. Its authentication setup is included so it can be run independently under the signed-in synthetic account. If a write times out, inspect current records before repeating it.

```
$profilePath = Join-Path $env:LOCALAPPDATA 'CivicSuite'
$sessionPath = Join-Path $profilePath 'config/local-session.json'
$session = Get-Content -LiteralPath $sessionPath -Raw | ConvertFrom-Json
$headers = @{ 'X-Townlight-Session' = $session.session_hash }
$body = @{
  title = 'Synthetic documentation exercise'
  scheduled_start = '2026-11-05T18:00:00-07:00'
  location = 'Synthetic room'
  meeting_type = 'regular'
} | ConvertTo-Json
$meeting = Invoke-RestMethod -Method Post `
  -Uri 'http://127.0.0.1:15480/desktop/v1/meetings/meetings' `
  -Headers $headers -ContentType 'application/json' -Body $body
Invoke-RestMethod `
  -Uri ('http://127.0.0.1:15480/desktop/v1/meetings/meetings/' + $meeting.id)
`
  -Headers $headers
Remove-Variable session, headers
```

This write requires Meetings to be enabled and the signed-in staff role to permit it. It is a cross-suite integration example; for this product's request models, use the complete technical reference below.

Complete technical contract

Read the Core API reference for every documented operation, request field, constraint, response declaration and source supplement. Download the OpenAPI JSON for tools. For Station this is

the suite-side schema only; its separate service contract is documented in the API supplement. The reference directory includes configuration/source inventories, dependency declarations, native command contracts, MCP signatures and candidate evidence.

MCP and automation boundary

No direct product MCP server is exposed by the accepted Townlight desktop or Station boundary. Do not configure an invented `townlight mcp` command or assume an AI client has staff permissions. The integration surfaces for this beta are native IPC, the documented HTTP APIs, source bundle imports, exact exports and the existing Station CLI.

The bundled transcription code includes a **TextFlowKit MCP adapter** as a separate source capability. It offers job-based `transcribe_media`, `get_transcript`, `list_jobs`, `get_job_status` and `export_transcript`, and additional tools declared in that adapter. It uses stdio first and optional Streamable HTTP, with the MCP SDK as an optional extra. It is not automatically launched, authenticated as Townlight staff or accepted as a Townlight product MCP deployment. This documentation does not add such a service.

For a transcription client already provisioned separately, an illustrative tool call is:

```
{"name":"get_job_status","arguments":{"job_id":"synthetic-existing-job-id"}}
```

Use a job ID returned by that transcription service. This example does not create a Townlight record or bypass its review/Records handoff. The exact optional adapter tool signatures are supplied in the MCP source reference. Station meeting capture does not become live speech recognition because this optional donor adapter exists.

Security and permissions

Use your own named account and save before leaving a screen. A saved draft is durable; unsaved form text may not be. If a write has an unknown result, reopen its record before repeating it.

Read source jurisdiction, dates, citation, version and visibility. A copied source hash identifies exact bytes, not their legal applicability. If a source changes or permission is removed, resolve the source problem rather than substituting stale cached content.

Keep originals and derivatives separate. Review the intended audience before export. A public summary excludes private fields only through the applicable copy contract and your review. A renamed private packet is still private content.

Drafting, review, approval, attachment, export, fulfillment and publication are separate operations. A Townlight workpaper does not itself adopt law, issue a permit, impose a fine or release a records response. The responsible staff member records the applicable decision.

The native session is validated and mapped to a persisted SQL identity. An editable actor/reviewer label is not authentication. Product/module permissions and source visibility apply at the server boundary. Revoked/unavailable authority fails closed for new Station operations; a previous permission result is not a perpetual authorization.

Use protected local folders, account permissions and credential storage. Never include the native session file, database passwords, provider keys or unredacted support bundles in a public issue or generated public documentation. A local public view is not automatically a secured internet deployment. Keep network exposure and real equipment acceptance specific to the chosen beta site.

Backup recovery and maintenance

Save work and use Core's canonical **Backup Now** before maintenance. That operation stops writers. Preserve the coherent profile/database/document backup and its receipt, then use **Start** to resume. A source export is not a full backup.

Use **Restore Latest Backup** through the supported controls. If the result says **Restore needs service start**, use **Start**, then **Check**, and reopen a known record. Do not repeat restore merely because services need to start. The fresh-profile acceptance verified that a fulfilled request survived restore and a post-backup marker did not.

Keep Station's dedicated database and media evidence together in its own backup. Core's native-profile backup must not be assumed to include an external Station data directory. Stop writers during a coordinated backup. Preserve failed media attempts and logs.

On failure, record the time, selected composition, component/installer versions and exact message. Check System Health for stopped, starting, ready or failed dependencies. Use the existing Retry control after readiness; do not create duplicate business records to refresh a stale view. Share only a redacted diagnostic bundle. Preserve secrets, original evidence, recovery material and unrelated installations.

Troubleshooting

Symptom	Likely meaning	Next action
Workspace absent	Wrong product composition or unavailable canonical runtime.	Check selected composition; activate through Finish Records Migration; wait for readiness.
Cannot sign in / 401	Missing or expired named account session.	Sign in normally; reopen a Station launch ticket if applicable.
Permission denied / 403	Account, module, source or browser-origin permission.	Ask the authorized administrator to inspect the intended permission; do not invent role headers.
Save conflict / 409	The saved revision, expected source hash or lifecycle changed.	Keep edits, reopen current state and reconcile deliberately.
Invalid input / 422	Required field, format, geometry, source envelope or state is invalid.	Read the field detail and correct the input/source; preserve originals.

Symptom	Likely meaning	Next action
Service unavailable / 503	Database, schema, connected source or configuration unavailable.	Inspect System Health and the named dependency. Preserve work; retry only after readiness.
Export refused	Missing exact review, approval, audience or source authorization.	Read the refusal and complete the owning product's explicit decision.
Restore needs service start	The snapshot was restored but services have not resumed.	Use Start, then Check and verify a saved record; do not blindly repeat restore.

Verified workflows and limits

Product	Recorded result	Precisely exercised
Core	PASS	Normal empty-profile setup; canonical activation; UI-created staff and sign-in; staff denied user administration; coherent backup, mutation, restore, Start/Check and retained/absent record readback.

These results come from the identified signed candidate's fresh normal-profile Sandbox exercise: real native IPC/HTTP/PostgreSQL and selected headless controls. All accounts, source bundles and media were synthetic. The guest install/uninstall and recovery passed, and the owned Sandbox stopped. No host installed application or standalone CivicCast was changed. This documentation task reuses those receipts; it does not claim a new software acceptance run.

The workflow matrix identifies raw receipt filenames. The evidence index retains their original hashes; raw receipts remain in the private beta handoff rather than being silently republished with the documentation site.

The limits remain: actual city field acceptance and equipment/network commissioning; Station automatic provisioning/SCM commissioning; every UI/role combination; field recognition accuracy; and historical in-place upgrades are not certified. They are not five invented feature requirements. Fresh-install controlled beta handoff remains the accepted scope. This package makes no city-deployment, publication or general-availability claim.

Terms of service and license

Scope and status

These terms describe the Townlight Beta 1 software and this documentation package. This is a locally prepared private beta, not a hosted-service subscription, a city deployment agreement or a public release. No uptime commitment, support response time, indemnity, data-processing

agreement or paid service is created by these documents. A separate signed agreement, if any, governs services it actually provides.

Software license and rights

Current Townlight-owned distributions identified in this candidate use **AGPL-3.0-only**. The full license is supplied with this documentation and with the source companion. The license governs rights to run, copy, modify and convey the covered software; these operational terms do not add restrictions to those rights. Historical grants and dependency licenses retain their own terms. CivicCast remains a separate product and is not relicensed by this documentation work.

When conveying covered software, retain applicable notices and provide the corresponding source and installation/build information required by the license. For a modified version supporting remote network interaction, follow section 13's source-offer requirement. The companion archive identifies the source for this candidate; a link to a different public branch is not a substitute for the matching source. Read the supplied license for the complete conditions. See the GNU license reference and GNU licensing FAQ.

Data ownership and access

Importing a document does not transfer its ownership or grant permission to publish it. The operating organization controls its accounts, access decisions, lawful source use, retention and review/release policy. Keep resident information, credentials, private notes and restricted evidence in their authorized locations. Documentation examples contain fictional data. No real city account, credential or record is required to exercise these examples in an isolated test environment.

Local operation and external processors

The accepted beta workflow is local and uses named staff accounts. Station has separate storage and credentials. Configure backup destinations and optional model/network providers deliberately. A configured external provider may receive submitted prompts, images or audio according to that configuration and provider's terms; this documentation does not promise that every optional configuration is offline. An email outbox or local public view does not establish external delivery or an internet deployment. This static documentation site has no analytics, account collection or remote font dependency.

Staff decisions and source applicability

Staff verify the jurisdiction, source currency, permissions, citations and proposed audience. Model output and generated guidance require review. No draft is automatically an official legal interpretation, zoning determination, permit approval, fee assessment, enforcement notice or records-release decision. Official permit fee authority remains with the adopted source and responsible organization; imported amounts do not become a new authoritative calculation.

Beta operation and recovery

Use the identified candidate in its supported fresh-install scope. Preserve backups and original evidence. Test physical equipment and chosen network/service configuration before relying on

them. The supplied acceptance describes selected synthetic workflows, not every screen, vendor, field condition or upgrade. Operators decide whether a particular deployment meets their own operational obligations.

Warranty and liability

The covered software's warranty and liability conditions are those in sections 15–17 of the supplied AGPL and any applicable separate written undertaking. This documentation creates no additional warranty or limitation beyond those terms. Third-party software is subject to its retained license notices. Nothing here removes rights that applicable law makes nonwaivable.

Support and incident reporting

Use the project's existing SUPPORT and SECURITY guidance for its matching repository. Report reproducible behavior with the version, operation and redacted logs. Do not put credentials, resident records or unredacted evidence in a public issue. Coordinate sensitive reports through the channel designated in SECURITY. No new support address or service commitment is invented here.

Ending a beta exercise and document changes

When an organization ends testing, it controls retention/export and removes access according to its policy. Uninstall may retain data; back up and inspect the selected removal options. Ending an exercise does not revoke rights already granted by AGPL. Later software/documentation versions must identify their changed candidate and scope; historical receipts retain their original identities. These terms do not claim that a city has accepted a contract merely by reading a manual.

Glossary

Term	Meaning
Approval	An explicit authorized decision on the saved text/artifact. It is separate from saving or attachment.
Canonical record	The owning product's retained identity and saved version, rather than an editable copied label.
Composition	The selected set of desktop modules; inspections-preview exposes the eight staff products.
Corresponding source	The source/build material matching the delivered covered program, subject to its license.
Derivative / Records copy	A separately prepared document with its own intended audience and hash.
Hash / SHA256	A fingerprint of exact bytes. It detects change; it does not establish legal authority.

Term	Meaning
Idempotency	Repeating the same identified request has the same intended effect; changing its payload can be refused.
IPC	The native desktop command boundary between its browser interface and Rust host.
MCP	Model Context Protocol; no direct Townlight product server is enabled by this beta.
Original	Retained source bytes. A reviewed derivative does not replace them.
Provenance	Source, version, date, actor and relationship information retained with evidence.
Revision	An immutable saved version. Later edits do not inherit an earlier review automatically.
Snapshot	A fixed audience-specific copy tied to a source/reviewed revision.
Station	Townlight’s separate media boundary using a pinned CivicCast-derived engine; standalone CivicCast remains separate.

Alphabetical subject index

- **Accounts and authentication** — Security and permissions.
- **AGPL and source** — Terms of service and license.
- **API contracts** — API and integration examples.
- **Architecture** — Architecture.
- **Backup** — Backup recovery and maintenance.
- **Beta version** — Beta identity.
- **Configuration** — Configuration and administration.
- **Evidence and acceptance** — Verified workflows and limits.
- **Fees and official decisions** — Terms of service and license.
- **First run** — First run.
- **Handoffs and audience** — Connected workflows.
- **MCP** — MCP and automation boundary.
- **Originals and provenance** — Security and permissions.
- **Recovery** — Backup recovery and maintenance.
- **Staff procedures** — Daily workflow.
- **Station configuration** — Configuration and administration.
- **Technical stack** — Technical specifications.
- **Troubleshooting** — Troubleshooting.

Operating subject links

- **Catalog import and readback** — Catalog import and readback.
- **Jurisdictional rule evaluation and review** — Jurisdictional rule evaluation and review.
- **Scaffold plan and generate** — Scaffold plan and generate.
- **Shared catalog rules and scaffolding** — Shared catalog rules and scaffolding.

Source and document control

This edition is authored from the immutable corresponding-source companion, the generated installed-domain schema and the retained workflow receipts. The current software versions and signed artifacts are unchanged. Public historical pages are not synchronized or published by this local documentation task.

Workflow subject index

- **Activate the runtime** — Daily workflow, step 2.
- **Collect diagnostics** — Daily workflow, step 7.
- **Complete setup** — Daily workflow, step 1.
- **Create staff access** — Daily workflow, step 3.
- **Import through the right product** — Daily workflow, step 4.
- **Make a coherent backup** — Daily workflow, step 5.
- **Restore and verify** — Daily workflow, step 6.
- Source inventory and file hashes
- All component metadata
- Installed domain OpenAPI
- Workflow matrix
- Full AGPL text